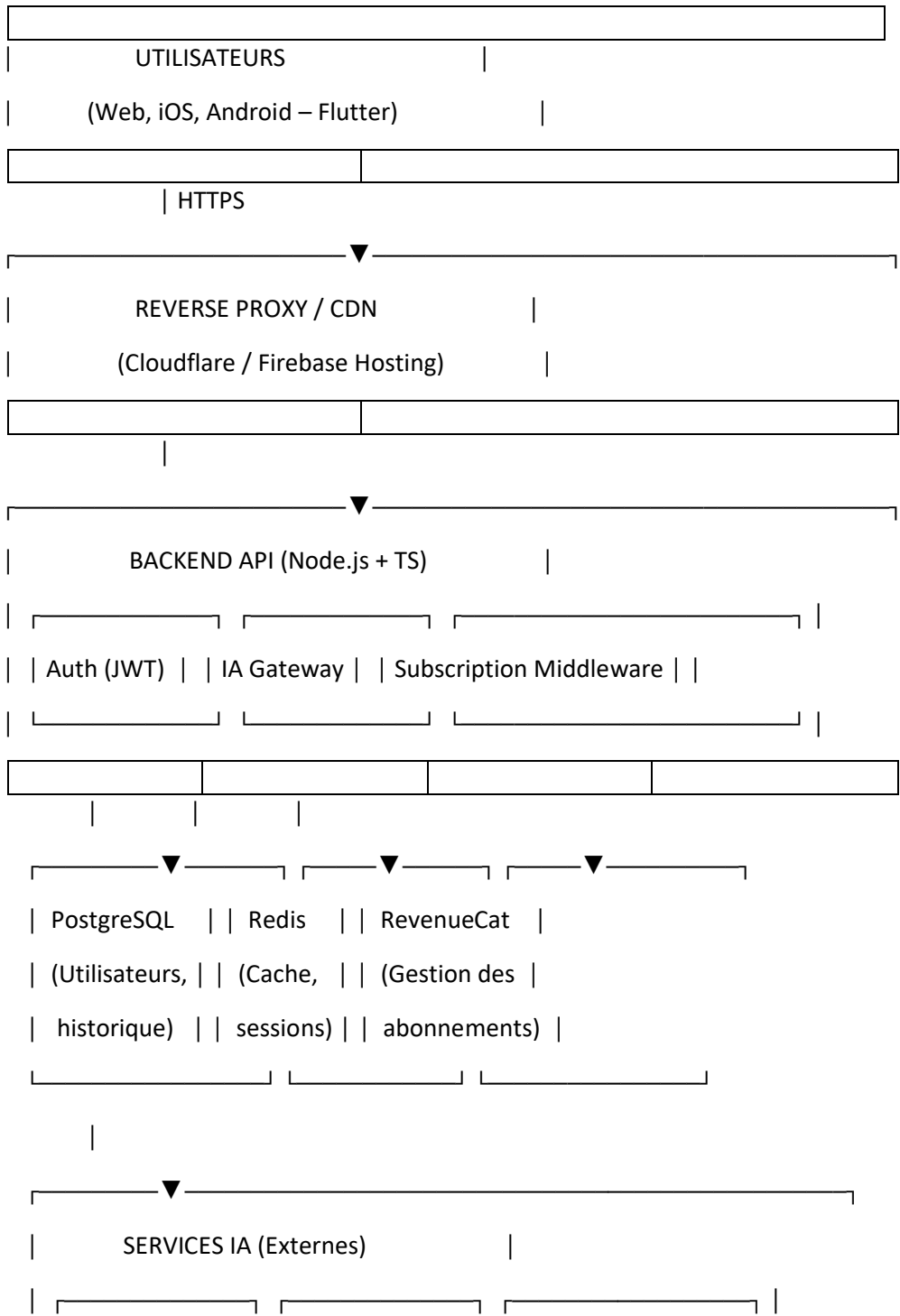
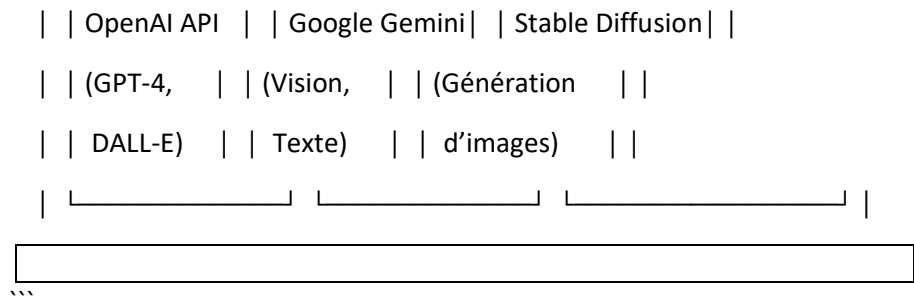


Voici la version finale complète de mon application de décoration avec assistant IA, système d'abonnement freemium et déploiement automatisé.

🏠 Architecture Globale

...



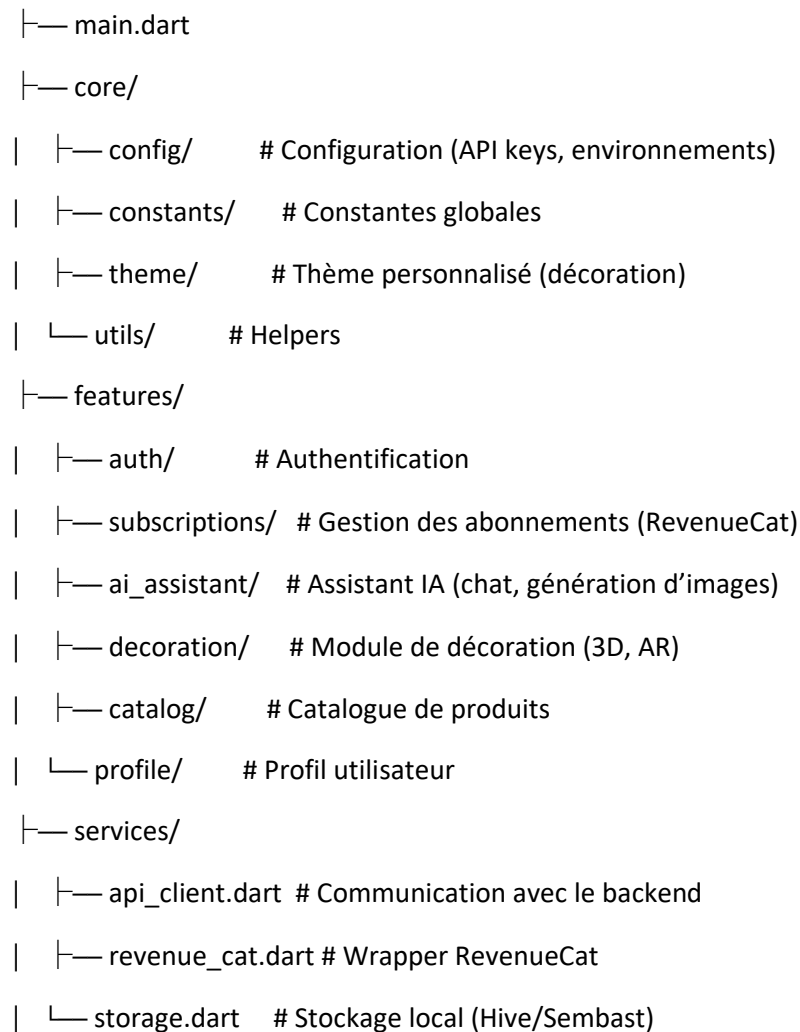


1. Frontend Flutter (Web, iOS, Android)

Structure du projet

...

Lib/



└─ widgets/ # Widgets réutilisables

...

Configuration des abonnements (RevenueCat)

```
```dart
```

```
// services/revenue_cat.dart
```

```
Import 'package:purchases_flutter/purchases_flutter.dart' ;
```

```
Class RevenueCatService {
```

```
 Static const String apiKey = 'VOTRE_CLE_REVENUECAT' ;
```

```
 Future<void> init() async {
```

```
 Await Purchases.setDebugLogsEnabled(true) ;
```

```
 Await Purchases.configure(PurchasesConfiguration(apiKey)) ;
```

```
 }
```

```
 Future<List<Package>> getPackages() async {
```

```
 Final offerings = await Purchases.getOfferings() ;
```

```
 Return offerings.current ?.availablePackages ?? [] ;
```

```
 }
```

```
 Future<void> purchasePackage(Package package) async {
```

```
 Await Purchases.purchasePackage(package) ;
```

```
 }
```

```
 Future<CustomerInfo> getCustomerInfo() async {
```

```
 Return await Purchases.getCustomerInfo() ;
```

```
 }
```

```
 Bool isSubscribed(CustomerInfo info) {
```

```

 Return info.activeSubscriptions.isNotEmpty ;
 }
}
...

```

Module Assistant IA

```

``dart
// features/ai_assistant/ai_chat.dart
Class AIChatScreen extends StatefulWidget {
 @override
 _AIChatScreenState createState() => _AIChatScreenState() ;
}

Class _AIChatScreenState extends State<AIChatScreen> {
 Final TextEditingController _controller = TextEditingController() ;
 Final List<Message> _messages = [] ;
 Bool _isLoading = false ;
 Int _dailyCount = 0 ;
 Final int _freeLimit = 5 ;

 Future<void> _sendMessage() async {
 Final text = _controller.text.trim() ;
 If (text.isEmpty) return ;

 // Vérifier le quota gratuit
 Final isSubscribed = await _checkSubscription() ;
 If (!isSubscribed && _dailyCount >= _freeLimit) {
 _showUpgradeDialog() ;
 Return ;
 }
 }
}

```

```

setState(() {
 _messages.add(Message(text : text, isUser : true)) ;
 _isLoading = true ;
});

Try {
 Final response = await ApiClient.sendAIMessage(text) ;
 setState(() {
 _messages.add(Message(text : response, isUser : false)) ;
 _dailyCount++ ;
 }) ;
} finally {
 setState(() => _isLoading = false) ;
}
}
}
...

```

---

## 2. Backend Node.js + TypeScript

### Structure

...

Backend/

```

├─ src/
| ├─ index.ts
| ├─ config/
| | ├─ database.ts
| | └─ redis.ts

```

```

| | └─ revenuecat.ts
| └─ middleware/
| | └─ auth.ts
| | └─ subscription.ts
| | └─ rateLimiter.ts
| └─ modules/
| | └─ auth/
| | └─ ai/
| | └─ subscription/
| | └─ decoration/
| └─ services/
| | └─ openai.ts
| | └─ revenuecat.ts
| | └─ storage.ts
| └─ types/
└─ Dockerfile
└─ docker-compose.yml
└─ .github/workflows/
| └─ deploy-backend.yml
| └─ deploy-frontend.yml
└─ package.json
...

```

Middleware de vérification d'abonnement

```

``typescript
// src/middleware/subscription.ts

Import { Request, Response, NextFunction } from 'express' ;

Import { getCustomerInfo } from '../services/revenuecat' ;

```

```

Export const requireSubscription = async (
 Req : Request,
 Res : Response,
 Next : NextFunction
) => {
 Try {
 Const userId = req.user.id ;
 Const customerInfo = await getCustomerInfo(userId) ;

 If (!customerInfo.activeSubscriptions.length) {
 // Vérifier le quota gratuit
 Const dailyUsage = await getDailyUsage(userId) ;
 If (dailyUsage >= 5) {
 Return res.status(402).json({
 Error : 'QUOTA_EXCEEDED',
 Message : 'Limite gratuite atteinte. Abonnez-vous pour continuer.'
 });
 }
 Req.isFree = true ;
 } else {
 Req.isFree = false ;
 }
 Next() ;
 } catch (error) {
 Res.status(500).json({ error : 'Subscription check failed' });
 }
};
...

```

Service IA avec fallback

```
``typescript
```

```
// src/services/openai.ts
```

```
Import OpenAI from 'openai' ;
```

```
Const openai = new OpenAI({ apiKey : process.env.OPENAI_API_KEY }) ;
```

```
Export async function generateDesignSuggestion(prompt : string, isPremium : boolean) {
```

```
 Const model = isPremium ? 'gpt-4-turbo' : 'gpt-3.5-turbo' ;
```

```
 Const maxTokens = isPremium ? 1000 : 300 ;
```

```
 Const response = await openai.chat.completions.create({
```

```
 Model,
```

```
 Messages : [
```

```
 { role : 'system', content : 'Tu es un expert en décoration intérieure...' },
```

```
 { role : 'user', content : prompt }
```

```
],
```

```
 Max_tokens : maxTokens,
```

```
 });
```

```
 Return response.choices[0].message.content ;
```

```
}
```

```
Export async function generateMoodboard(description : string, isPremium : boolean) {
```

```
 If (!isPremium) {
```

```
 Throw new Error('La génération d\'images est réservée aux abonnés') ;
```

```
 }
```

```
 Const response = await openai.images.generate({
```

```
 Model : 'dall-e-3',
```

```
 Prompt : `Moodboard de décoration : ${description}`,
```

```
 N : 1,
```



```
 Size : '1024x1024',
 });

 Return response.data[0].url ;
}
...
```

---

### 3. Déploiement Automatisé (CI/CD)

Workflow GitHub Actions pour le Backend

```
``yaml
.github/workflows/deploy-backend.yml
Name : Deploy Backend

On :
 Push :
 Branches : [main]
 Paths :
 - 'backend/**'

Env :
 REGISTRY : ghcr.io
 IMAGE_NAME : ${{ github.repository }}/backend

Jobs :
 Test-and-build :
 Runs-on : ubuntu-latest
 Steps :
 - Uses : actions/checkout@v4
```

- Name : Setup Node.js

Uses : actions/setup-node@v4

With :

Node-version : '20'

Cache : 'npm'

Cache-dependency-path : backend/package-lock.json

- Name : Install dependencies

Run : npm ci

Working-directory : backend

- Name : Run tests

Run : npm test

Working-directory : backend

- Name : Build

Run : npm run build

Working-directory : backend

- Name : Log in to Container Registry

Uses : docker/login-action@v3

With :

Registry : \${{ env.REGISTRY }}

Username : \${{ github.actor }}

Password : \${{ secrets.GITHUB\_TOKEN }}

- Name : Build and push Docker image

Uses : docker/build-push-action@v5

With :

Context : backend

Push : true

Tags : \${{ env.REGISTRY }}/\${{ env.IMAGE\_NAME }} :latest,\${{ env.REGISTRY }}/\${{ env.IMAGE\_NAME }} :\${{ github.sha }}

Deploy :

Needs : test-and-build

Runs-on : ubuntu-latest

Steps :

- Name : Deploy to Render

Run : |

Curl -X POST \${{ secrets.RENDER\_DEPLOY\_HOOK }}

...

Workflow GitHub Actions pour le Frontend Flutter

```yaml

.github/workflows/deploy-frontend.yml

Name : Deploy Flutter App

On :

Push :

Branches : [main]

Paths :

- 'lib/**'
- 'pubspec.yaml'

Jobs :

Build-and-deploy-web :

Runs-on : ubuntu-latest

Steps :

- Uses : actions/checkout@v4

- Name : Setup Flutter

Uses : subosito/flutter-action@v2

With :

Flutter-version : '3.22.0'

Channel : 'stable'

- Name : Install dependencies

Run : flutter pub get

- Name : Analyze code

Run : flutter analyze

- Name : Run tests

Run : flutter test

- Name : Build web

Run : flutter build web --release --base-href= »/ »

- Name : Deploy to Firebase Hosting

Uses : FirebaseExtended/action-hosting-deploy@v0

With :

repoToken : \${{ secrets.GITHUB_TOKEN }}

firebaseServiceAccount : \${{ secrets.FIREBASE_SERVICE_ACCOUNT }}

channelId : live

projectId : votre-projet-firebase

build-android :

runs-on : ubuntu-latest

steps :

- uses : actions/checkout@v4

- name : Setup Flutter

Uses : subosito/flutter-action@v2

With :

Flutter-version : '3.22.0'

- Name : Build Android

Run : |

Flutter build appbundle --release

Flutter build apk --release

- Name : Upload to Play Store (via fastlane)

Run : |

Cd android

Fastlane deploy

Build-ios :

Runs-on : macos-latest

Steps :

- uses : actions/checkout@v4

- name : Setup Flutter

Uses : subosito/flutter-action@v2

With :

Flutter-version : '3.22.0'

- Name : Build iOS

Run : flutter build ios --release --no-codesign

- Name : Upload to App Store (via fastlane)

Run : |

Cd ios

Fastlane deploy

...

Dockerfile Backend

```
```dockerfile
```

```
backend/Dockerfile (multi-stage)
```

```
FROM node :20-alpine AS builder
```

```
WORKDIR /app
```

```
COPY package*.json ./
```

```
RUN npm ci
```

```
COPY . .
```

```
RUN npm run build
```

```
FROM node :20-alpine
```

```
WORKDIR /app
```

```
COPY --from=builder /app/dist ./dist
```

```
COPY --from=builder /app/node_modules ./node_modules
```

```
COPY package*.json ./
```

```
EXPOSE 3000
```

```
USER node
```

```
CMD [« node », « dist/index.js »]
```

```
```
```

Docker Compose (Développement)

```
```yaml
```

```
docker-compose.yml
```

```
Version : '3.8'
```

```
Services :
```

```
 Backend :
```

```
 Build : ./backend
```

```
 Ports :
```

```
 - « 3000 :3000 »
```

Environment :

- NODE\_ENV=development
- DATABASE\_URL=postgresql://user :pass@postgres :5432/decoration
- REDIS\_URL=redis://redis :6379

Depends\_on :

- postgres
- redis

Postgres :

Image : postgres :15

Environment :

- POSTGRES\_USER=user
- POSTGRES\_PASSWORD=pass
- POSTGRES\_DB=decoration

Volumes :

- Pgdata:/var/lib/postgresql/data

Ports :

- « 5432 :5432 »

Redis :

Image : redis :7-alpine

Ports :

- « 6379 :6379 »

Volumes :

Pgdata :

...

## Étapes d'installation

### Étape Action Outil

- 1 Configurer les comptes Firebase, Render/Heroku, RevenueCat, OpenAI
- 2 Initialiser le projet Flutter flutter create
- 3 Configurer RevenueCat Ajouter SDK, créer les offres
- 4 Développer le backend Node.js + TypeScript + PostgreSQL
- 5 Configurer les secrets GitHub API keys, tokens
- 6 Mettre en place les workflows CI/CD GitHub Actions
- 7 Déployer en staging flutter build web --release
- 8 Tester les paiements Sandbox RevenueCat
- 9 Déployer en production git push main

### Variables d'environnement à configurer

```
```.env

# Backend

DATABASE_URL=postgresql://...

REDIS_URL=redis://...

OPENAI_API_KEY=sk-...

REVENUECAT_API_KEY=rc_...

JWT_SECRET=...


# Frontend (via --dart-define)

REVENUECAT_API_KEY=rc_...

API_BASE_URL=https://api.votreapp.com

...
```



5. Résumé Exécutif

Composant Technologie Hébergement

Frontend Flutter 3.22+ Firebase Hosting (Web), Play Store/App Store (Mobile)

Backend Node.js + TypeScript Render.com / AWS ECS (conteneurisé)

Base de données PostgreSQL 15 Render.com / AWS RDS

Cache Redis 7 Render.com / AWS ElastiCache

Paielements RevenueCat + Stripe API RevenueCat

IA OpenAI API (GPT-4, DALL-E) API externe

CI/CD GitHub Actions Automatisé sur push main

Monitoring Sentry + Logtail SaaS

Prochaines Actions Immédiates

1. Créez un dépôt GitHub et structurez les dossiers (frontend/, backend/)
 2. Configurez Firebase pour l'hébergement web et l'authentification
 3. Créez un compte RevenueCat et configurez vos offres (Mensuel, Trimestriel, Annuel)
 4. Obtenez vos clés API (OpenAI, Stripe, RevenueCat)
 5. Ajoutez les secrets dans les paramètres GitHub Actions
 6. Poussez le code → le déploiement se fera automatiquement
-

Si vous souhaitez que je détaille un composant spécifique (par exemple, l'intégration 3D pour la décoration, le module de réalité augmentée, ou la configuration exacte de fastlane pour les stores), n'hésitez pas à me le préciser.